Assignment 2: Concrete Architecture of OpenCV

October 31, 2025

Website : https://softwarearchitecture.wixsite.com/eecs4314assignment

Group Name: SourceArchitects

Team Members Information

Name	Student ID	Email
Sheida Khodabakhshian Khansari	215534340	sheidakh@my.yorku.ca
Vrinda Ashwar	218624692	vrinda03@my.yorku.ca
Matthew Lee	218256743	mattl7@my.yorku.ca
Werner Bagu-Paiz	21727772	wernerbp@my.yorku.ca
William Moran	218928960	willvm24@my.yorku.ca
Stefewn Johnson	219411099	stef9@my.yorku.ca

Abstract **2**
1) Introduction and Overview **2**
2) Concrete Architecture **2**
2.1) Derivation Process 2
2.2) LSEdit Architecture Figures 5
2.3) Overview of Derived Architecture 7
2.3) Architectural Styles 8
2.4 Design Patterns 8
3) Image Processing Subsystems & Interactions **9**
3.1) Drawing 9
3.2) Color_ops (Color Space Conversions) & ColourMap 9
3.4) Geometric Image Transformations(Geometry) 11
3.5) misc (Miscellaneous Image Transformations) 11
3.6) subdivision(planar subdivision) 11
3.7) histogram 11
3.10) Features(Feature Detection) 12
3.11) Core_internals: 12
4) Conceptual Architecture vs. Concrete Architecture (Reflexion analysis) **13**
4.1) Conceptual & Reflexion Architecture: 13
4.2) Divergences & Discrepancies: 14
5) Sequence Case Example **14**
6) Lessons Learned **15**
7) Conclusion **15**
8) References **15**

Abstract

The following report provides an overview of the concrete architecture of OpenCV, a widely used open-source computer vision library. Building on our previous analysis of its conceptual architecture, this report focuses on the internal, practical structure of OpenCV's design, highlighting how its subsystems are implemented and interact in practice. It specifically examines the image processing subsystem (imgproc), one of OpenCV's core modules that supports operations such as image filtering, geometric transformations, color processing, morphology, segmentation, and structural analysis. The concrete architecture was derived by extracting and transforming OpenCV's file dependencies using tools such as Understand, Grok, and LSEdit. Automated scripts were developed to group files into hierarchical subsystems, allowing clearer visualization of module relationships within LSEdit. The analysis reveals that OpenCV's concrete design follows a layered modular architecture, consisting of an upper, middle, and lower layer. In addition, several design patterns such as Facade, Strategy, Factory, and Template were identified within the imgproc subsystem, contributing to the system's extensibility and maintainability. Finally, a comparison between the conceptual and concrete architectures was conducted, revealing key differences between the two. Overall, this report highlights how OpenCV's implementation reflects its modular design and illustrates the interactions among its various subsystems.

1) Introduction and Overview

The OpenCV (Open Source Computer Vision Library) is a powerful open-source library of programming functions with the main goal of real-time computer vision [1]. OpenCV, a highly used software system in academia and industry, is a complex and mature software system with the architectural integrity of which is essential for its performance and maintainability. This report will give an overview of the concrete architecture of the OpenCV library, mainly focusing on one of the top-level subsystems, the image processing subsystem (imgproc) [2]. Being one of the most basic and commonly used modules of the OpenCV library, the imgproc module offers the necessary algorithms on which image filtering, geometric transformations, color space transformations, and morphological operations are based [2]. Understanding the structural composition of the subsystem, determining the main architectural styles it implements, and shedding light on interesting design solutions that enable its high functionality are achieved through reverse engineering with the help of SciTools Understand. The discussion indicates that the modular decomposition and dependency control of the imgproc library enable the capability and usefulness for an extensive range of computer vision tasks, and enable high-performance at the same time as real-time applications.

2) Concrete Architecture

2.1) Derivation Process

As shown in figure 1 below, the concrete architecture of OpenCV 4.12.0 was recovered by transforming flat file dependencies into a hierarchical subsystem model for LSEdit. Understand by SciTools was used to parse the source code and exported opencv-4_FileDependencies.csv, containing thousands of files. A Perl script

(transformUnderstand.pl) converted this to opencv-4_FileDependencies.raw.ta into tuple-attribute (TA) format with $INSTANCE and $cLinks entries.

With the help of Grok, generate_contain.py was developed to automate containment. It parses .raw.ta, strips opencv-4.12.0/, filters src/include/build, and infers the subsystem hierarchy. It generates two outputs: opencv-4_FileDependencies.contain with containment lines and an .instances file containing $INSTANCE … cSubSystem declarations to be appended to the raw TA file after the first line. This automation ensured consistency, eliminated duplicates, and scaled efficiently to the full dataset.

The hierarchical decomposition was then integrated with the raw TA file using the lab-provided createContainment script (.bat or .sh), producing opencv-4_FileDependencies.ls.ta. This file enabled visualization in LSEdit, where subsystem dependencies were rendered as an interactive graph.

Initial graphs were dense, so it was refined in place within the same .contain file, adding sub-subsystems under imgproc.ss (e.g., geometry, filter) based on function clusters. Throughout the refinement loop, Grok served as an AI-assisted reviewer. Grok validated logic, found orphans, and guided refinement via dependency and naming analysis. Given OpenCV's scale, manual containment was infeasible, so automation ensured accuracy. Grok enabled rapid, informed iteration.

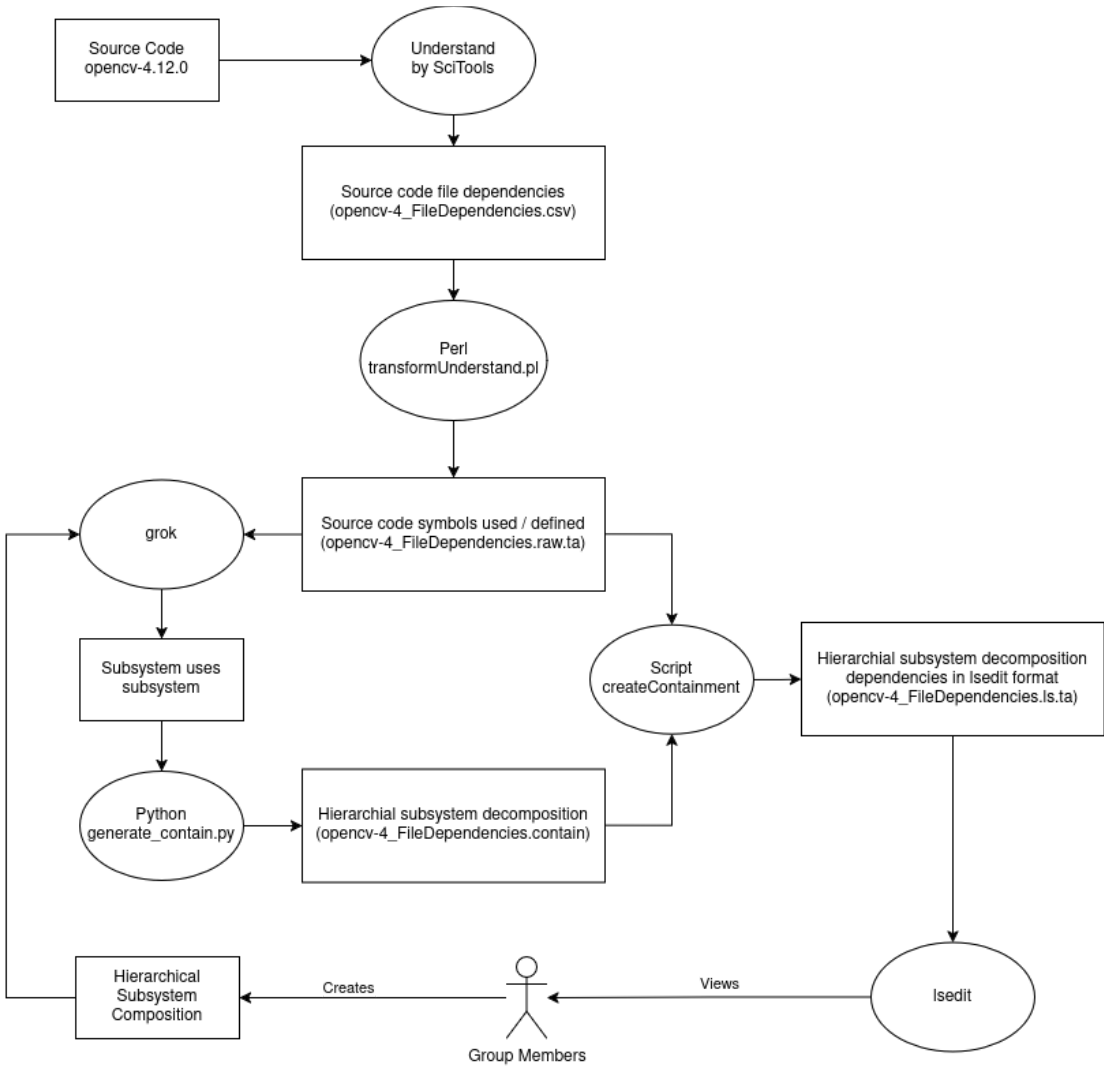


Figure 1. Derivation Process for Concrete Architecture from Source Code.

2.2) LSEdit Architecture Figures

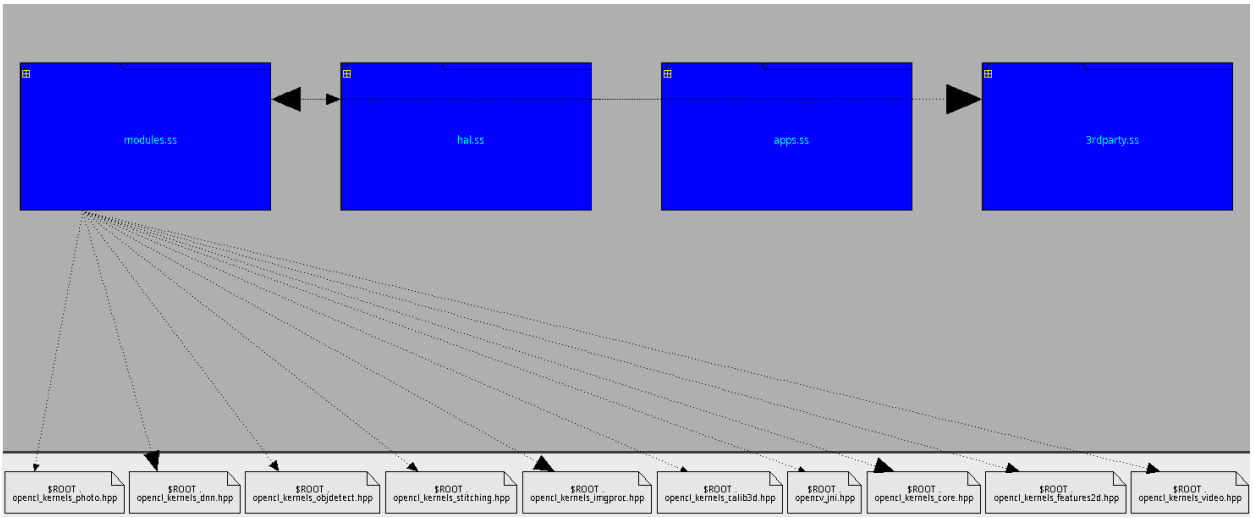


Figure 2. High level architecture of OpenCV captured from LSEdit.

For better resolution, please view this link: highLevel.png

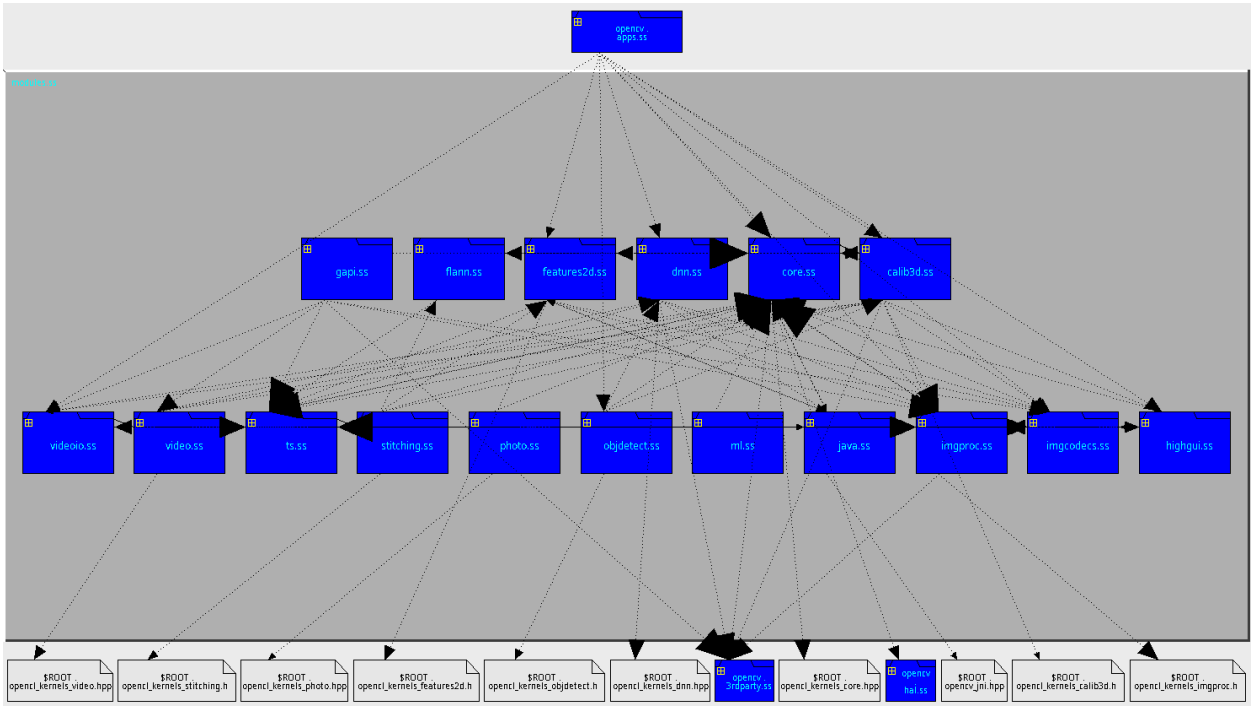


Figure 3. High level architecture of "Module" captured from LSEdit.

For better resolution, please view this link: module.png

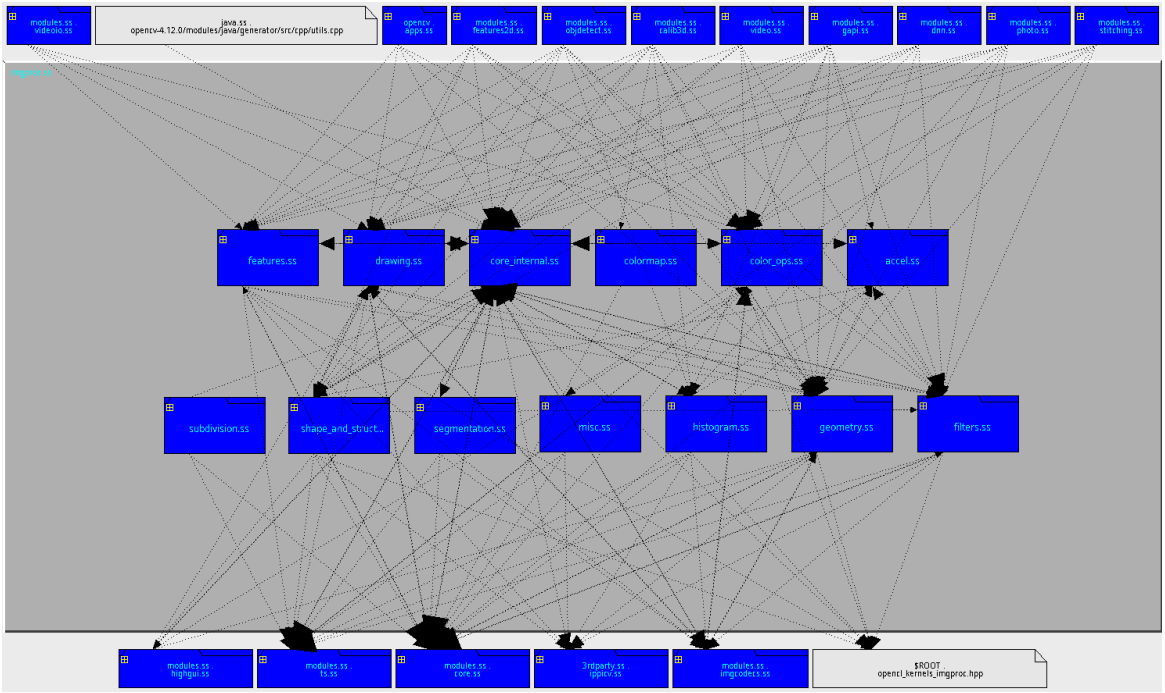


Figure 4. Architecture of "imgproc" module captured from LSEdit.

For better resolution, please view this link: imgproc.png

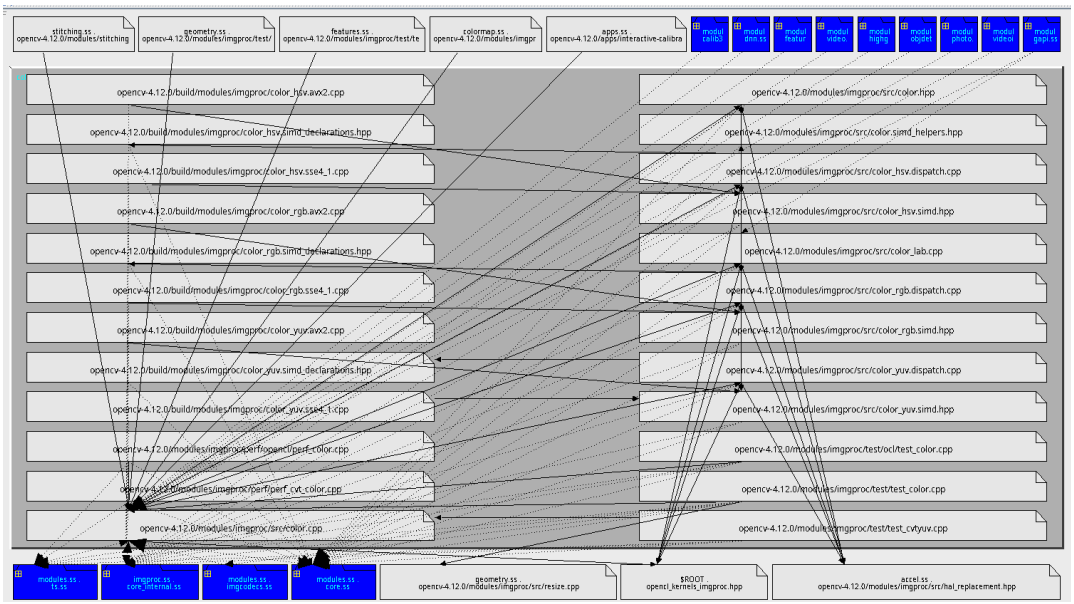


Figure 5. Architecture of "color_ops" which represents the colour operations subsystem of "imgproc" captured from LSEdit.

For better resolution, please view this link: colour_op.png

2.3) Overview of Derived Architecture

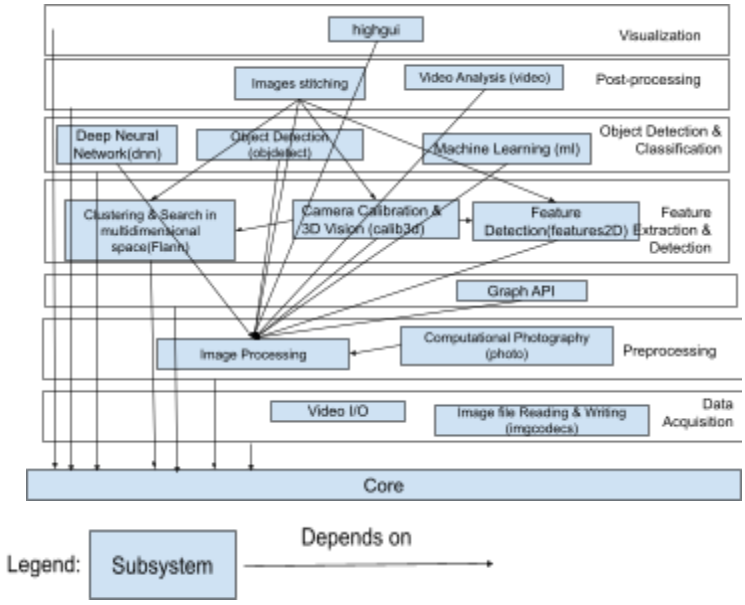


Figure 6. Conceptual architecture of OpenCV.

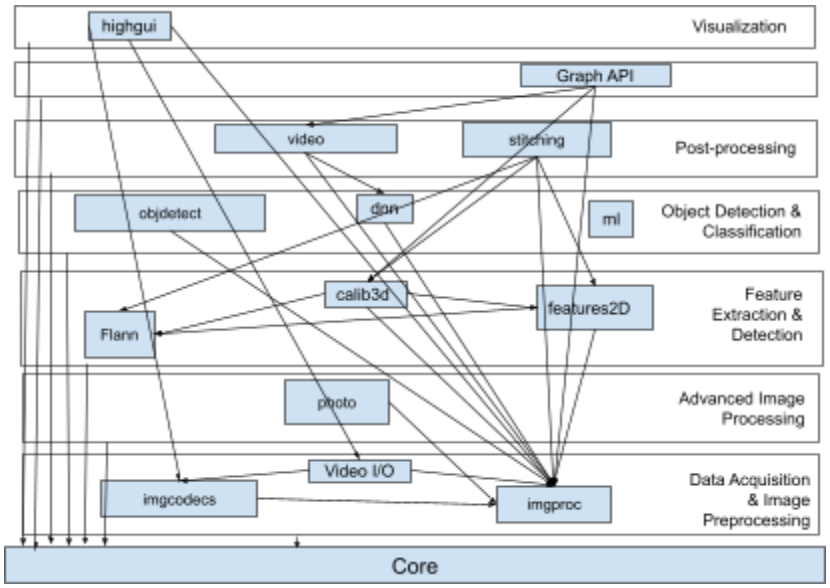


Figure 7. Derived concrete architecture of "module" from figure 3.

Following the derivation process, the conceptual architecture shown in figure 6 was modified based on existing dependencies found in figure 4 to derive the concrete architecture as shown in figure 7.

As shown in figure 2, the high level architecture of OpenCV contains the "module" subsystem and the HAL subsystem. The hardware acceleration layer (HAL) provides an abstraction layer that allows OpenCV modules to delegate performance-critical operations to optimized hardware-specific implementations without creating direct dependencies on specific hardware backends [3-5]. The module subsystem is further broken down into the following subsystems: highgui, gapi, video, stitching, objdetect, dnn, ml, flann, calib3d, features2D, photo, videoio, imgcodecs and imgproc. The overview of the functionalities of these modules were provided in Assignment1 report.

Furthermore, as shown in figure 4, the overall architecture of OpenCV is a layered architecture where all modules depend on the "core" module. The majority of the modules also depend on the image processing module (imgproc). The architecture of imgproc is shown in figure 4. This architecture will further be explored in this report.

2.3) Architectural Styles

From the diagrams, as shown in figure 7, it can be observed that OpenCV follows a layered modular architectural style, with the image processing subsystem located in the middle layer. The upper layer includes modules such as highgui, which provide user-facing interfaces and rely on imgproc to perform operations like filtering and color conversion [6]. The imgproc module, which is in the middle layer, serves as the main algorithmic component of OpenCV's image processing pipeline. It focuses on implementing the computational logic, applying transformations, feature extraction, and image manipulation routines [6]. The lower layer consists of foundational modules such as core and hal, which provide the mathematical operations, matrix representations, and hardware abstraction that imgproc depends on [6]. By structuring these modules in a layered manner this allows for both extensibility and maintainability across the system.

2.4 Design Patterns

The imgproc subsystem uses four design patterns to give OpenCV major advantages in simplicity, flexibility, and maintainability. The first is the Facade pattern, which provides a simple interface to a complex system. In imgproc, it acts as a facade where you call a simple function, and it hides the internal complexity of creating filters, handling image borders, and using optimized code. This enables the API to be convenient to work with without having to read the complex mathematics within. The second design pattern is the Strategy pattern, which lets you select an algorithm from a family of options at runtime[7]. The imgproc uses it for filtering, interpolation, and color conversion. This is done by the specific strategy being chosen based on the flag or parameters. The Factory pattern is also used to provide an interface for creating objects, letting subclasses decide which exact class to initiate [8]. In imgproc, it uses this pattern to create algorithm implementations or wrappers depending on the environment. This makes the system more flexible and easier to extend. The Template pattern is also used in the system, which defines the skeleton of an algorithm in a base class, allowing subclasses to override specific steps without changing the overall structure[9]. These four design patterns make the OpenCV system more extensible using either factory or strategy patterns to easily add new algorithms, and the Facade pattern to make it easier to use, hiding complex functions.

3) Image Processing Subsystems & Interactions

The architecture of "imgproc" as shown in figure 4, was derived from examining the code documentation and using the derivation process outlined in section 2.1. According to OpenCV's documentation, "imgproc" modules contains image processing functions that enable tasks such as image filtering, geometric image transformations, miscellaneous image transformation, drawing functions, color space conversions, colour maps, planar subdivision, histograms, structural analysis and shape descriptors, motion analysis and object tracking, feature detection, object detection, and image segmentation [10]. Thus it can be organized into several distinct subsystems each handling a specific category of image processing operations.

It is important to note that both "object detection" and "Motion Analysis and Object Tracking" which are listed as modules under "imgproc" in OpenCV documentation, do not contain any actual function implementations in imgproc. There were no files included in these two submodules so they were not added to the final LSEdit figure of the "imgproc" architecture in figure 4. Both @defgroup imgproc_object and @defgroup imgproc_motion were included in the imgproc module header file but they likely serve as a documentation placeholder for organizational purposes.

The derived subsystems for "imgproc" based on documentation and file dependencies were the following; features, drawing, colormap, color_ops, subdivision, shape_and_structural_analysis,segmentation, misc which stands for "Miscellaneous Image Transformations" , histogram, geometry and filters. Core_internals, as well as accel which stands for the "Hardware Acceleration Layer" were also included in the architecture as these subsystems depend on them. Below is the overview of each subsystem of imgproc.

3.1) Drawing

The "drawing" subsystem provides a set of functions for applying geometric shapes, text, and contours onto images. These functions work with matrices or images of arbitrary depth and support antialiasing for 8-bit images. These functions include basic shapes like line, arrowedLine, rectangle, circle, ellipses as well as polygon functions like fillPoly() which fills the area bounded by one or more polygons[11],[12].

3.2) Color_ops (Color Space Conversions) & ColourMap

The color_ops subsystem is dedicated to converting images between different color representations and analyzing color space conversion operations. Color_ops subsystem contains operations that enable programmers to convert pictures in one color format to another, eg, RGB to Grayscale[13]. These conversions are also necessary in image analysis processes such as segmentation, tracking objects, and object detection, where certain color models are used to find it easier to recognize patterns or regions of interest. The subsystem is based on regular mathematical transformations, which are described in model modules of OpenCV and are performed with optimized C++ functions[14]. The color functions are principally found in module/imgproc/src/color.cpp and typically work together with the Filter and Geometry subsystems to make images ready to be used in other processing[14].

ColourMap subsystem provides visualization enhancement by mapping grayscale intensities to colors using predefined color lookup tables (LUTs), however it doesn't convert between different colour formats; the underlying data representation remains unchanged. Only the information is displayed with colour for better visualizations[15].

3.3) Image Filtering (filters)

"Filters" subsystem contains classes and functions [16]. These classes and functions can be executed on 2D images in two different types of filtering operations [16]. It can be done linearly and nonlinearly [16]. While doing filtering, every pixel that's in a source image will be processed by checking its neighbourhood [17]. After checking, it will start calculating a brand new value [16]. When we are doing linear filtering, the calculation is done a little differently [16]. A weighted sum of neighboring pixel values is included in the calculation [16]. We also have another type of operation called morphological [16]. How this operation works is that it searches for the maximum or the minimum value that's in the area we want to search in [16]. There are also many types of methods to do border extrapolation that is given by OpenCV [16]. This is needed when filtering [16]. The reasons are that when we are filtering, there are pixels that are not within the boundaries of the image [16]. They are outside of it [16]. So we need something to handle that case [16]. So some of the methods of border extrapolation are to be able to fill constant values in an area that doesn't exist or reproduce some of the border pixels [16]. This is done so the output is more accurate [16]. There are many filtering functions within OpenCV [16]. If we want to smooth an image then the best functions to use for this would be boxFilter(), GaussianBlur(), and blur() [16]. If you wanted to maintain the edges and reduce noise in the image, then the function to accomplish this would be the medianBlur() function [16]. There also another function called bilateralFilter() that's best for edge-preserving smoothing [17]. There are also filters that are derivative based [16]. Some examples of these filters are Scharr(), Sobel(), and Laplacian() [17]. What these filters do is that they highlight gradients and edges [16]. There are also other operations like dilate(), morphologyEx(), and erode() [16]. These are morphological operations that's job is to change the structure of the image so that it's features are either emphasized or suppressed [16]. All these functions are essential for their own purpose for image processing [16].

Morphological transformations are part of the filtering subsystem which are operations like Erosion, Dilation, Opening and Closing, etc. These image processing techniques analyze and modify images based on shape using a structuring element or kernel to operate on pixel neighbourhoods. They have two inputs; the original image and the second is the structuring element or kernel which determines the nature of the operation. The two main morphological operators are Erosion and Dilation. Erosion involves the shrinking of the foreground objects while Dilation involves expanding them. The Opening operation involves erosion followed by dilation to remove noise and the Closing operation is dilation followed by erosion to close gaps. Morphological gradient also provides the difference between dilation and erosion of an image while "Top Hat" is the difference between input image and the opening of the image and "Black Hat" is the difference between the closing of the input image and the input image[17].

3.4) Geometric Image Transformations(Geometry)

The Geometry functions in OpenCV's imgproc module are used to change the shape or position of an image without changing its actual content [18]. They work by deforming the pixel grid and mapping those pixels into a new layout [18]. Instead of copying pixels forward, these functions look at each pixel in the output and find where it came from in the original image [18]. This helps avoid visual errors when transforming the image [18]. Overall, the Geometry subsystem helps transfer images while keeping the original details intact [18].

3.5) misc (Miscellaneous Image Transformations)

The misc subsystem which stands for the Miscellaneous Image Transformations group contains image processing functions that don't fit in other categories like filtering, geometric transforms, or color conversions. This group includes thresholding operations, distance transform operations, watershed segmentation and other transformations that don't belong to more specialized groups like geometric transformation or filtering[19].

3.6) subdivision(planar subdivision)

The "subdivision" subsystem which stands for "planar subdivision" performs various planar subdivision on set of 2D points. It allows for a plane to be subdivided into triangles using the Delunary's algorithms.The subdivisions can be used for the 3D piece-wise transformation of a plane, morphing, fast location of points on the plane, building special graphs[20].

3.7) histogram

The histogram subsystem provides functions for calculating, comparing and analyzing histograms. These histograms represent image intensity distributions and these functions are thus useful for understanding contrast, brightness and intensity distribution[21].

3.8) Segmentation

The Segmentation subsystem in OpenCV's imgproc module divides an image into regions or objects [22]. One of the key functions is grabCut, which separates foreground and background using a mask and an initial rectangle [22]. Another function is watershed, which performs marker-based segmentation to grow regions from seeds [22]. In the watershed method you first mark regions or seeds, using contour detection, then the algorithm assigns every pixel to a region or a boundary [22]. Overall, the Segmentation subsystem transforms a raw image into discrete, labeled components that follow structure and object boundaries [22].

3.9) shape_and_structural_analysis (Structural Analysis and Shape Descriptors)

"Shape_and_structural_analysis" subsystem which handles structural analysis and shape descriptors. It contains functions for analyzing and describing geometric properties of shapes and contours in images as well as performing shape matching and comparison. Unlike the "feature" subsystem it works on already extracted contours[23].

3.10) Features(Feature Detection)

The features subsystem which stands for “feature detection” provides functions useful for computer vision tasks like object recognition and image matching. These functions can detect distinctive points, edges, lines, and circles in images. This subsystem provides basic feature detection for corners, edges and lines but the features2d module provides more advanced feature detection and description algorithms.This subsystem’s goal is to identify, examine, and refine the distinctive points on the image [24]. These points can be lines, edges, and corners [24]. This is what explains an image’s structure [24]. In this, we have the function Canny() that detects edges [24]. We also have the Hough transforms that detect circles and lines [24]. There are functions like goodFeaturesToTrack(), preCornerDetect(), Harris, and MinEigenVal, which are all corner-based [24]. The cornerSubPix is a refinement function [24]. There is also a LineSegmentDetector whose job is to give advanced line detection control [24]. It also gives sub-pixel accuracy [24]. There are feature descriptors that enable key points to be matched across images and allow them to be explained [24]. FREAK, BRIEF, and ORB would be feature descriptors [24]. With all of this, it makes a strong structure for vision tasks at the high levels [24]. This includes motion tracking, shape analysis, and object recognition [24].

3.11) Core_internals:

The Core subsystem serves as the foundation of OpenCV, providing essential data structures and computational operations that many other modules rely on. One of these modules is imgproc which relies on the Core module for handling matrix operations, mathematical computations and memory management [25]. For example, when imgproc applies filters or performs transformations, Core module’s cv::Mat is used to store and modify that image data [25]. The Core subsystem acts as the backbone of OpenCV as it plays an important role in other modules' usability as seen with imgproc.

4) Conceptual Architecture vs. Concrete Architecture (Reflexion analysis)

4.1) Conceptual & Reflexion Architecture:

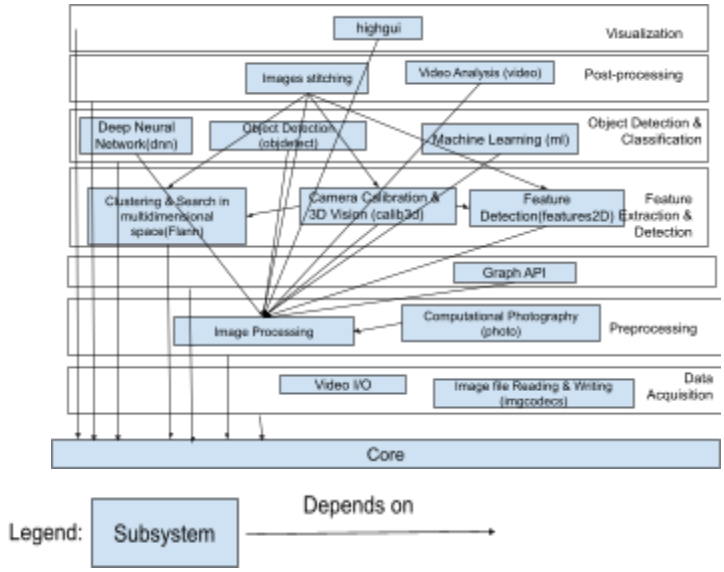


Figure 8. Conceptual Architecture

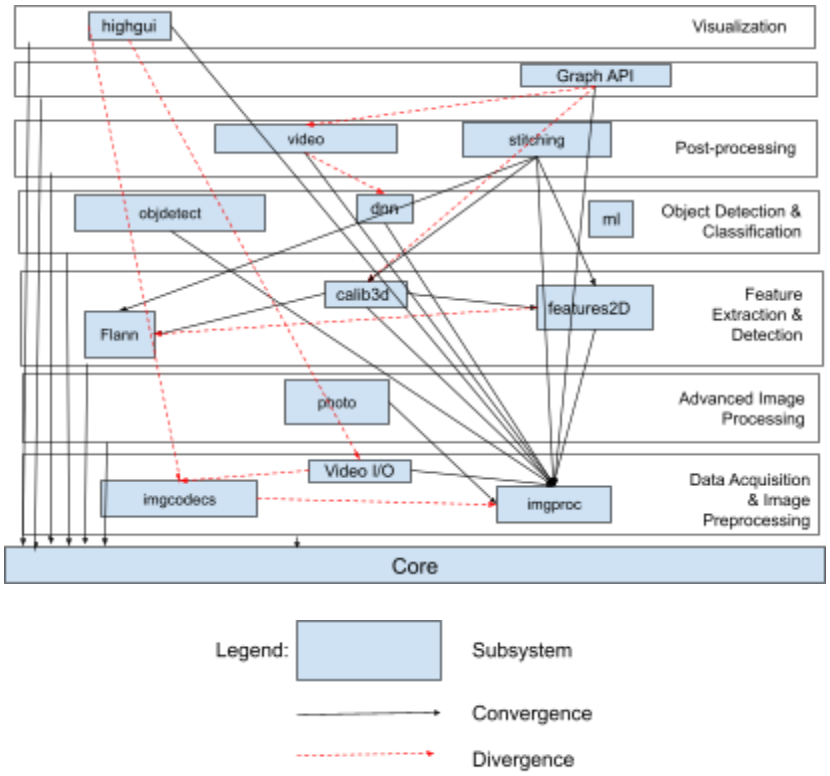


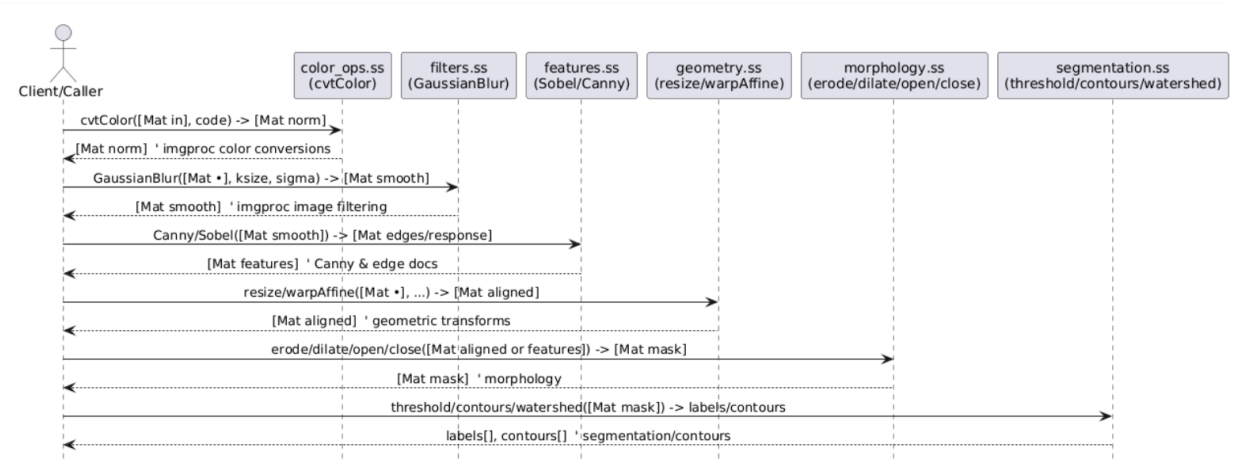
Figure 9. Reflexion Architecture

4.2) Divergences & Discrepancies:

Several differences were found between the conceptual and concrete architecture. As seen in figure 9, several dependencies were missing. With the exception of the dependency of videoio module on imgcodecs and imgcodecs dependency on imgproc, most of these missing dependencies were optional modular dependencies as found through examining the configuration build files (CMakeLists.txt) for these modules. Also the Graph-API (gapi) module was moved to an upper layer as it has an optional dependency on the video module.Image processing was also moved to a lower layer as both imagecodecs and videoio modules depend on this module.

Furthermore, contrary to the conceptual diagram, the module ml doesn't depend on the imgproc module. According to the configuration file for this model CMakeLists, this module only depends on the core module based on the following line; "ocv_define_module(ml opencv_core WRAP java objc python)".

5) Sequence Case Example



This shows how an image moves through the imgproc subsystem step-by-step. The image starts as a core::Mat, and each imgproc module performs one type of processing on it [26]. First, color_ops.ss can change the color format using cvtColor [13]. Next, filters.ss smooths the image using GaussianBlur to remove noise [27]. Then, features.ss can find edges using functions like Canny or Sobel [28]. geometry.ss then changes the image size or shape with functions such as resize or warpAffine [18]. If a mask is needed, morphology.ss cleans it up using operations like erode or dilate [11]. Finally, segmentation.ss extracts objects or regions using findContours or threshold [17]. In every step, the same Mat data type is passed along, showing how each subsystem in imgproc handles a different part of the image-processing system.

6) Lessons Learned

By researching on the image processing subsystem, it gave us some valuable experience. We learned how to understand and recover software architectures that have a large scale. When we created the custom containment file, we noticed that are understanding towards it improved alot. We learned that this is a very important step when we are trying to decompose structure of the subsystem. Since there was a lot of requirements for this project in a short time period, we noticed that communication and teamwork was very important in order to get this project done. Also we learned that by using the architectural patterns that it made managing complexity much easier. Therefore to conclude, we learned the importance of having a well established architecture and a modular design. It makes high complex systems way easier to scale and maintain.

7) Conclusion

In conclusion, the concrete architecture analysis of OpenCV revealed a layered modular structure. Using tools like Understand, Grok, and LSEdit, OpenCV's internal dependencies were constructed and visualized. Specifically, the imgproc subsystem showed how its components such as filters, geometry, color, morphology, and segmentation work together through a shared to form complete image processing operations. The identified design patterns, including Facade, Strategy, Factory, and Template, show how OpenCV makes complex tasks easier while still allowing new features to be added.Comparing the conceptual and concrete architectures also shows how OpenCV's structure has developed over time. Overall, this study showed the importance of organizing the system into clear parts and using automation to better understand, maintain, and document a large software project like OpenCV.

8) References

[1]"OpenCV: OpenCV modules," *Opencv.org*, 2025. https://docs.opencv.org/4.12.0/

[2] "OpenCV: Image Processing," *Opencv.org*, 2025. https://docs.opencv.org/4.12.0/d7/dbd/group__imgproc.html (accessed Oct. 27, 2025).

[3] "Hardware abstraction layer (HAL) overview | Android Open Source Project." Accessed: Oct. 31, 2025. [Online]. Available: https://source.android.com/docs/core/architecture/hal

[4] "OpenCV: Hardware Acceleration Layer." Accessed: Oct. 31, 2025. [Online]. Available: https://docs.opencv.org/3.4/d3/df3/group__imgproc__hal.html

[5] P. Nelson, "Introducing HAL riscv-rvv: Unleashing the power of RISC-V CPUs with RVV 1.0," OpenCV. Accessed: Oct. 31, 2025. [Online]. Available: https://opencv.org/blog/introducing-hal-riscv-rvv-unleashing-the-power-of-risc-v-cpus-with-rvv-1-0/

[6] OpenCV Documentation, https://docs.opencv.org/4.x/ (accessed Oct. 28, 2025)

[7] Refactoring.Guru, "Strategy Pattern," 2023. [Online]. Available: https://refactoring.guru/design-patterns/strategy. [Accessed: Oct. 28, 2024].

[8] Refactoring.Guru, "Factory Method Pattern," 2023. [Online]. Available: https://refactoring.guru/design-patterns/factory-method. [Accessed: Oct. 28, 2024]

[9] Refactoring.Guru, "Template Method Pattern," 2023. [Online]. Available: https://refactoring.guru/design-patterns/template-method. [Accessed: Oct. 28, 2024].

[10]"OpenCV: Image Processing." Accessed: Oct. 06, 2025. [Online]. Available: https://docs.opencv.org/4.x/d7/dbd/group__imgproc.html

[11] “OpenCV: Drawing Functions.” Accessed: Oct. 31, 2025. [Online]. Available: https://docs.opencv.org/4.x/d6/d6e/group__imgproc__draw.html
[12] “OpenCV: Contours : Getting Started.” Accessed: Oct. 31, 2025. [Online]. Available: https://docs.opencv.org/3.4/d4/d73/tutorial_py_contours_begin.html
[13]“Color conversions,” OpenCV, https://docs.opencv.org/3.4/de/d25/imgproc_color_conversions.html (accessed Oct. 31, 2025).
[14] opencv, “opencv/modules/imgproc/src/color.cpp at 4.x · opencv/opencv,” *GitHub*, 2025. https://github.com/opencv/opencv/blob/4.x/modules/imgproc/src/color.cpp (accessed Oct. 28, 2025).
[15]“OpenCV: ColorMaps in OpenCV.” Accessed: Oct. 31, 2025. [Online]. Available: https://docs.opencv.org/4.x/d3/d50/group__imgproc__colormap.html
[16]“Image Filtering,” OpenCV, https://docs.opencv.org/4.12.0/d4/d86/group__imgproc__filter.html (accessed Oct. 28, 2025).
[17]“Morphological Transformations,” OpenCV, https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html (accessed Oct. 31, 2025).
[18] “OpenCV: Geometric Image Transformations,” docs.opencv.org. https://docs.opencv.org/3.4/da/d54/group__imgproc__transform.html (accessed Oct. 31, 2025).
[19] “OpenCV: Miscellaneous Image Transformations.” Accessed: Oct. 31, 2025. [Online]. Available: https://docs.opencv.org/4.x/d7/d1b/group__imgproc__misc.html
[20] “OpenCV: Planar Subdivision.” Accessed: Oct. 31, 2025. [Online]. Available: https://docs.opencv.org/4.x/df/d5b/group__imgproc__subdiv2d.html
[21] “OpenCV: Histograms.” Accessed: Oct. 31, 2025. [Online]. Available: https://docs.opencv.org/4.x/d6/dc7/group__imgproc__hist.html

[22] https://docs.opencv.org/4.x/d3/d47/group__imgproc__segmentation.htmlOpenCV, "Image processing module (imgproc) API reference," OpenCV Foundation, 2023. [Online]. Available: https://docs.opencv.org/4.12.0/d7/dbd/group__imgproc.html. [Accessed: Oct. 25, 2023
[23]“OpenCV: Structural Analysis and Shape Descriptors.” Accessed: Oct. 31, 2025. [Online]. Available: https://docs.opencv.org/4.x/d3/dc0/group__imgproc__shape.html
[24]“Feature Detection,” OpenCV, https://docs.opencv.org/4.12.0/dd/d1a/group__imgproc__feature.html (accessed Oct. 28, 2025).
[25] “Core functionality (core module),” OpenCV Documentation, accessed Oct. 28, 2025. [Online]. Available: https://docs.opencv.org/4.x/d0/de1/group__core.html
[26] “CV::Mat class reference,” OpenCV, https://docs.opencv.org/3.4/d3/d63/classcv_1_1Mat.html (accessed Oct. 31, 2025).
[27]“Smoothing Images,” OpenCV, https://docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html (accessed Oct. 31, 2025).
[28] “Feature detection,” OpenCV, https://docs.opencv.org/4.x/dd/d1a/group__imgproc__feature.html (accessed Oct. 31, 2025).